

Lab No 1: Implementation and Analysis of Sorting Algorithms – Insertion Sort (Unit 1)

1. Objective and Real-World Application

Objective: To implement the Insertion Sort algorithm and analyze its time complexity under different scenarios (best, average, and worst cases).

Real-World Application: Insertion sort is highly efficient for nearly sorted data, making it ideal for real-time applications where data arrives incrementally, such as keeping a live scoreboard or a retail transaction history sorted.

2. Pre-requisites (Micro-topics)

- Array traversal and element shifting mechanics.
- The concept of in-place sorting and stable sorting.
- Asymptotic notations ($O(n)$ vs $O(n^2)$) and loop invariant properties.

3. Brief Description and Tools

Step 1: Divide the array logically into a sorted and an unsorted part.

Step 2: Iterate from the second element to the last element (key).

Step 3: Compare the current element ("key") with its predecessor. If the key is smaller, shift the predecessor elements to the right until the correct position for the key is found.

Step 4: Insert the key and repeat until the array is fully sorted.

Tools: Python language only. Editors like VS Code, PyCharm, and Google Colab.

4. Descriptive Questions

Level 1: Understanding and Implementation

Question: Provide a clean Python implementation of Insertion Sort and explain how the inner loop mechanically shifts elements to insert the key in its correct relative position.

Level 2: Applying Concepts

Question: Trace the step-by-step array state modification during Insertion Sort for the input array [12, 11, 13, 5, 6]. Specify the state of the array after every full pass of the outer loop.

Level 3: Advanced Challenge

Question: Mathematically demonstrate why the best-case time complexity of Insertion Sort is $O(n)$ while its worst-case is $O(n^2)$. Describe an operational scenario where Insertion Sort systematically outperforms Quick Sort.